```
"""
Project 3: Random Password Generator
(DecodeLabs | Python Programming)

Input-Process-Output scaffold:
    Phase 1  Input    - validate the
requested password length
    Phase 2  Process  - pool characters
from `string`, select with `secrets`,
                        assemble with
''.join() (linear time, no `+=` in a loop)
    Phase 3  Output   - report entropy (E =
L * log2(R)) with the password
"""

import math
import secrets
import string

MIN_LENGTH = 8       # hard floor (the
brief's example length)
RECOMMENDED = 15     # NIST SP 800-63B-4
minimum for single-factor passwords
MAX_LENGTH = 64      # NIST: systems must
accept at least 64 characters


# ---------- Phase 1: input and validation
----------
def get_length() -> int:
```

```
    """Keep asking until the user enters a
whole number in the allowed range."""
    while True:
        raw = input(f"Password length
({MIN_LENGTH}-{MAX_LENGTH}): ").strip()
        try:
            length = int(raw)
        except ValueError:
            print("  Please enter a whole
number, e.g. 16.")
            continue
        if MIN_LENGTH <= length <=
MAX_LENGTH:
            return length
        print(f"  Length must be between
{MIN_LENGTH} and {MAX_LENGTH}.")


def ask_yes_no(prompt: str) -> bool:
    """Ask a y/n question until a valid
answer is given."""
    while True:
        answer =
input(prompt).strip().lower()
        if answer in ("y", "yes"):
            return True
        if answer in ("n", "no"):
            return False
        print("  Please answer y or n.")


# ---------- Phase 2: transformation engine
----------
def get_pools(use_symbols: bool) ->
```

```
list[str]:
    """Character groups from the string
module (no hand-typed alphabets)."""
    pools = [string.ascii_letters,
string.digits]
    if use_symbols:
        pools.append(string.punctuation)
    return pools


def generate_password(length: int, pools:
list[str]) -> str:
    """Build a password with at least one
character from every pool."""
    full_pool = "".join(pools)

    # One guaranteed character per pool,
the rest from the combined pool.
    chars = [secrets.choice(pool) for pool
in pools]
    chars += [secrets.choice(full_pool) for
_ in range(length - len(chars))]

    # Shuffle with the OS entropy source so
guaranteed characters don't
    # always sit at the front
(random.shuffle would be predictable).
    secrets.SystemRandom().shuffle(chars)
    return "".join(chars)


# ---------- Phase 3: entropy validation --
--------
def entropy_bits(length: int, pool_size:
```

```
int) -> float:
    """E = L * log2(R). Upper-bound
estimate: assumes uniform selection."""
    return length * math.log2(pool_size)


def rate(bits: float) -> str:
    """Rough strength label (heuristic
thresholds)."""
    if bits < 60:
        return "Weak"
    if bits < 80:
        return "Fair"
    if bits < 100:
        return "Strong"
    return "Very strong"


def main() -> None:
    print("=== Random Password Generator
===")

    length = get_length()
    if length < RECOMMENDED:
        print(f"  Note: NIST recommends
{RECOMMENDED}+ characters.")
    use_symbols = ask_yes_no("Include
symbols (@, #, $ ...)? [y/n]: ")

    pools = get_pools(use_symbols)
    password = generate_password(length,
pools)
    bits = entropy_bits(length,
len("".join(pools)))
```

```
    print(f"\nPassword : {password}")
    print(f"Length   : {length}")
    print(f"Entropy  : {bits:.1f} bits
({rate(bits)})")


if __name__ == "__main__":
    try:
        main()
    except (KeyboardInterrupt, EOFError):
        print("\nCancelled.")
```